

SpringMVC 核心

Http 消息转换器

什么是 HTTP 消息转换器

1. HTTP 消息指的是请求协议的内容和响应协议的内容。
2. HTTP 消息转换器是：它将HTTP请求和响应中的原始数据（如JSON/XML/String）与Java对象之间进行智能双向转换，让开发者能够直接处理业务对象，不需要手动解析和组装数据格式。
3. 比如，前端提交一个 json 字符串，后端通过消息转换器直接将 json 字符串转换成 java 对象
4. 或者后端生成了 java 对象，消息转换器也可以自动将 java 对象转换成 json 格式的字符串响应给前端。

HTTP 消息转换器需要我们写吗

不需要，SpringMVC 框架已经内置了很多消息转换器，可以满足日常开发，除非极特殊情况，内置的消息转换器无法完成时，我们可以自己定制。

内置常用的 HTTP 消息转换器有哪些

1. 所有的 HTTP 消息转换器都有一个公共的接口：`HttpMessageConverter`
2. 该接口下常见的消息转换器包括：
 - a. `StringHttpMessageConverter`（请求和响应阶段都有用）
 - b. `MappingJackson2HttpMessageConverter`（请求和响应阶段都有用）
 - c. `FormHttpMessageConverter`（主要在请求阶段起作用）

怎么指定使用哪个 HTTP 消息转换器

在程序当中通过编写不同的注解，来选择不同的 HTTP 消息转换器。

在响应阶段：（使用`@ResponseBody`来启用消息转换器）

1. 使用 `@ResponseBody` 注解来启用 `StringHttpMessageConverter` 和 `MappingJackson2HttpMessageConverter`
2. 当使用 `@ResponseBody` 并且 `Controller#method()` 返回一个普通的字符串时：`StringHttpMessageConverter` 启用。它会把字符串直接写入响应体，假设方法返回 `hello` 字符串，最终浏览器页面上就显示一个 `hello` 字符串。
3. 当使用 `@ResponseBody` 并且 `Controller#method()` 返回一个 `Java` 对象时：`MappingJackson2HttpMessageConverter` 启用（底层优先使用 `jackson` 来解析 `json`）。它会将 `Java` 对象转换成 `JSON` 格式的字符串响应给前端。
4. 在响应阶段如果没有使用 `@ResponseBody` 注解标注，则默认会走 `ModelAndView` 机制，不走消息转换器机制。
5. 总结一句话：如果需要响应 `JSON` 给前端，就需要在 `Controller#method()` 上添加 `@ResponseBody` 注解，并返回一个 `Java` 对象（一般是返回一个统一的 `R` 对象。）。。

在请求阶段：（`@RequestBody`来启用消息转换器）

1. 当请求头的 `Content-Type: application/json`，并且 `Controller#method()` 方法的参数上使用了 `@RequestBody`，那么前端提交的 `JSON` 字符串将被转换成 `Java` 对象，底层使用的消息转换器是：`MappingJackson2HttpMessageConverter`
2. 当请求头的 `Content-Type` 是 `text/plain`，并且 `Controller#method()` 方法的参数上使用了 `@RequestBody`，后端会使用 `StringHttpMessageConverter`
3. 当请求头的 `Content-Type` 是 `application/x-www-form-urlencoded` 或 `multipart/form-data`，并且 `Controller#method()` 方法的参数上使用了 `@RequestBody`，后端会使用 `FormHttpMessageConverter`。

4. 如果 `Controller#method()` 方法的参数上没有添加 `@RequestBody` 注解，是不会走任何消息转换器的。
5. 总结一句话：前端如果提交 JSON 字符串，后端 `Controller#method()` 参数设置为 Java 对象，并让参数使用 `@RequestBody` 注解进行标注。底层会自动将 JSON 字符串转换成 Java 对象。

对于响应来说 mv 不是 null 时不会使用消息转换器

对于响应来说，如果 `DispatcherServlet` 的 `doDispatch` 方法中的 `mv` 不是 `null`，则响应时不会走任何消息转换器。

ModelAndView 和 **消息转换器** 是 Spring MVC 中两条完全不同的响应处理路径，它们互斥。

ModelAndView 机制中，当请求提交数据，后端进行数据绑定时使用的机制是：**WebDataBinder 机制**。

@ResponseBody 的使用

@ResponseBody 可以出现的位置

```
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface ResponseBody {
}
```

出现在类上 方法上

1. 出现在方法上：只作用于当前方法。
2. 出现在类上：作用于当前类中所有的方法。

功能一：响应普通字符串到前端

第一步：前端 axios 发送 ajax post 请求

- test.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>test</title>
</head>
<body>
  <button id="LoadMsgBtn">获取消息</button>
  <div id="msg"></div>
  <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
  <script>
    document.addEventListener("DOMContentLoaded", function(){
      let loadMsgBtn = document.querySelector("#LoadMsgBtn");
      let msgDiv = document.querySelector("#msg");
      loadMsgBtn.addEventListener("click", async function (){
        try{
          // 发送ajax post请求
          let response = await axios.post("msg");
          msgDiv.textContent = response.data;
        }catch (e) {
          console.log(e);
        }
      });
    });
  </script>
</body>
</html>
```

```
</script>
</body>
</html>
```

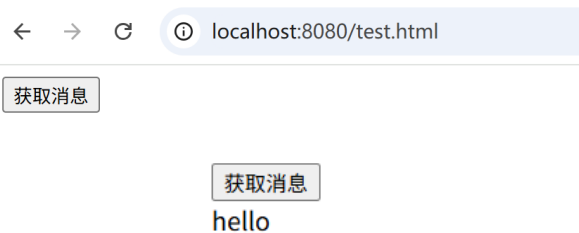
第二步：编写 Controller，使用@ResponseBody注解，响应普通字符串

```
package org.cqipu.edu.controller;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.ResponseBody;

@Controller
public class ResponseBodyController {

    @PostMapping("/msg")
    @ResponseBody
    public String hello(){
        // 重点是这里：由于使用@ResponseBody注解，因此返回值不再被当做逻辑视图名
        // 底层会使用消息转换器StringHttpMessageConverter，将该字符串直接写入响应体
        return "hello";
    }
}
```

第三步：测试：<http://localhost:8080/test.html>



总结：使用@ResponseBody，`return "hello";`不再是逻辑视图名了，是一个普通的字符串，底层将自动使用 `StringHttpMessageConverter` 转换器，将字符串直接写入响应体。

功能二：响应 json 字符串到前端

第一步：引入解析 JSON 的库：jackson的依赖

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.0</version>
</dependency>
```

第二步：编写实体类

```
package org.cqipu.edu.entity;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class User {
    private Long id;
```

```

    private String name;
    private String email;
    private Integer gender;
}

```

第三步：编写 Controller

```

package org.cqipu.edu.controller;
import org.cqipu.edu.entity.User;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.ResponseBody;

@Controller
public class ResponseBodyController {

    @PostMapping("/msg")
    @ResponseBody
    public User hello() {
        User user = new User(100L, "张三", "zhangsan@123.com", 1);
        return user;
    }
}

```

第四步：编写 axios 发送 ajax post 请求

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>test</title>
</head>
<body>
<button id="LoadMsgBtn">获取消息</button>
<div id="msg"></div>
<script src="https://unpkg.com/axios/dist/axios.min.js"></script>
<script>
    document.addEventListener("DOMContentLoaded", function(){
        let loadMsgBtn = document.querySelector("#LoadMsgBtn");
        let msgDiv = document.querySelector("#msg");
        loadMsgBtn.addEventListener("click", async function (){
            try{
                // 发送ajax post请求
                let response = await axios.post("msg");
                msgDiv.textContent = JSON.stringify(response.data); // 返回了一个对象，将对象转换成json字符串
            }catch (e) {
                console.log(e);
            }
        });
    });
</script>
</body>
</html>

```

第五步：测试

获取消息

```
{ "id":100,"name":"张三","email":"zhangsan@123.com","gender":1}
```

总结：使用 `@ResponseBody` 注解标注，并且 `return obj;` 时，另外也引入了解析 `json` 的 `java` 库之后，会自动使用 `MappingJackson2HttpMessageConverter` 消息转换器。

@RestController 的使用

1. 现代开发中，一般 `Controller` 中每一个方法上都需要添加 `@ResponseBody` 注解，因为大部分情况下都是返回 `JSON` 字符串给前端，为了注解复用，`@ResponseBody` 最好写在类上面，如下：

```
@Controller
@ResponseBody
public class ResponseBodyController {
    @PostMapping("/msg1")
    public String hello1() {
        return "hello";
    }
    @PostMapping("/msg2")
    public User hello2() {
        User user = new User(100L, "张三", "zhangsan@123.com", 1);
        return user;
    }
}
```

2. 以上代码可以继续再简化，`SpringMVC` 提供了 `@RestController`，`@RestController = @Controller + @ResponseBody`，因此代码直接这样写即可：

```
@RestController
public class ResponseBodyController {
    @PostMapping("/msg1")
    public String hello1() {
        return "hello";
    }
    @PostMapping("/msg2")
    public User hello2() {
        User user = new User(100L, "张三", "zhangsan@123.com", 1);
        return user;
    }
}
```

@RequestBody 的使用

`@RequestBody` 经常使用在：前端提交一个 `JSON` 字符串。后端要将 `JSON` 字符串转换成 `Java` 对象。

```
@Target(ElementType.PARAMETER)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface RequestBody {
    // 它只允许出现在参数上
    boolean required() default true;
}
```

- 第一步：引入 `jackson` 依赖
- 第二步：编写 `User` 类
- 第三步：编写 `Controller`



```
@RestController
public class ResponseBodyController {
    @PostMapping("/msg")
    public User hello(@RequestBody User user) {
        return user;
    }
}
```

第四步：编写 axios 发送 ajax post 请求



```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>test</title>
</head>
<body>
<button id="LoadMsgBtn">获取消息</button>
<div id="msg"></div>
<script src="https://unpkg.com/axios/dist/axios.min.js"></script>
<script>
    document.addEventListener("DOMContentLoaded", function(){
        let loadMsgBtn = document.querySelector("#LoadMsgBtn");
        let msgDiv = document.querySelector("#msg");
        loadMsgBtn.addEventListener("click", async function (){
            try{
                // 前端准备好数据
                let data = {
                    id: 1,
                    name: "admin",
                    gender: 1,
                    email: "admin@123.com"
                };
                // 发送ajax post请求
                let response = await axios.post("msg", data);
                msgDiv.textContent = JSON.stringify(response.data); // 返回了一个对象，将对象转换成json字符串
            }catch (e) {
                console.log(e);
            }
        });
    });
</script>
</body>
</html>
```

第五步：测试

获取消息

```
{"id":1,"name":"admin","email":"admin@123.com","gender":1}
```

RequestEntity

RequestEntity不是一个注解，是一个普通的类。这个类的实例封装了整个请求协议：**包括请求行、请求头、请求体所有信息。**

出现在控制器方法的参数上：

- UserController.java



```
@RequestMapping("/send")
@ResponseBody
public String send(RequestEntity<User> requestEntity){
    System.out.println("请求方式: " + requestEntity.getMethod());
    System.out.println("请求URL: " + requestEntity.getUrl());
    HttpHeaders headers = requestEntity.getHeaders();
    System.out.println("请求的内容类型: " + headers.getContentType());
    System.out.println("请求头: " + headers);

    User user = requestEntity.getBody();
    System.out.println(user);
    System.out.println(user.getUsername());
    System.out.println(user.getPassword());
    return "success";
}
```

在实际的开发中，如果你需要获取更详细的请求协议中的信息。可以使用 `RequestEntity`

ResponseEntity

`ResponseEntity`不是注解，是一个类。用该类的实例可以封装响应协议，包括：状态行、响应头、响应体。也就是说：如果你想定制属于自己的响应协议，可以使用该类。

假如我要完成这样一个需求：前端提交一个id，后端根据id进行查询，如果返回null，请在前端显示404错误。如果返回不是null，则输出返回的user。

- UserController.java



```
@Controller
public class UserController {

    @GetMapping("/users/{id}")
    public ResponseEntity<User> getUserById(@PathVariable Long id) {
        User user = userService.getUserById(id);
        if (user == null) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body(null); // 404
        } else {
            return ResponseEntity.ok(user); // 200
        }
    }
}
```

测试：当用户不存在时



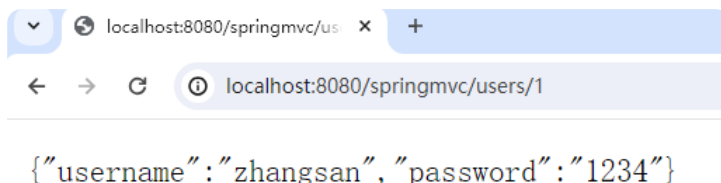
找不到 localhost 的网页

找不到与以下网址对应的网页: <http://localhost:8080/springmvc/users/1>

HTTP ERROR 404

重新加载

测试：当用户存在时



RESTful 的 AJAX 请求总结

1. 发送 get 请求: `axios.get(url, config)`，如果需要提交多个参数，则在 `config` 对象中配置 `params` 属性，例如以下代码：

```
● ● ●
axios.get('/api/users', {
  params: {
    id: 1,
    name: "zhangsan"
  }
});
```

2. 发送 post 请求: `axios.post(url, data, config)`
3. 发送 put 请求: `axios.put(url, data, config)`
4. 发送 delete 请求: `axios.delete(url, config)`，删除多条数据时，则在 `config` 对象中配置 `params` 属性，例如以下代码：

```
● ● ●
axios.delete('/api/users', {
  params: {
    ids: [2, 3] // 使用数组
  }
});
```


文件上传和下载

文件上传

前端页面：

- index.html

```
● ● ●
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>文件上传</title>
</head>
<body>

<!-- 文件上传表单-->
<form th:action="@{/file/up}" method="post" enctype="multipart/form-data">
  文件: <input type="file" name="fileName"/><br>
  <input type="submit" value="上传">
</form>

</body>
</html>
```

重点是：form表单采用post请求，enctype是multipart/form-data，并且上传组件是：type="file"

web.xml文件：

- web.xml

```
● ● ●
<!-- 前端控制器-->
<servlet>
  <servlet-name>dispatcherServlet</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>classpath:springmvc.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
  <multipart-config>
    <!-- 设置单个支持最大文件的大小-->
    <max-file-size>102400</max-file-size>
    <!-- 设置整个表单所有文件上传的最大值-->
    <max-request-size>102400</max-request-size>
    <!-- 设置最小上传文件大小-->
    <file-size-threshold>0</file-size-threshold>
  </multipart-config>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcherServlet</servlet-name>
  <url-pattern>/</url-pattern>
</servlet-mapping>
```

重点：在DispatcherServlet配置时，添加 multipart-config 配置信息。

Controller中的代码：

```
● ● ●
```

```

package org.cqipu.edu.controller;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.multipart.MultipartFile;

import java.io.*;
import java.util.UUID;

@Controller
public class FileController {

    @RequestMapping(value = "/file/up", method = RequestMethod.POST)
    public String fileUp(@RequestParam("fileName") MultipartFile multipartFile, HttpServletRequest request) throws
IOException {
        String name = multipartFile.getName();
        System.out.println(name);
        // 获取文件名
        String originalFilename = multipartFile.getOriginalFilename();
        System.out.println(originalFilename);
        // 将文件存储到服务器中
        // 获取输入流
        InputStream in = multipartFile.getInputStream();
        // 获取上传之后的存放目录
        File file = new File(request.getServletContext().getRealPath("/upload"));
        // 如果服务器目录不存在则新建
        if(!file.exists()){
            file.mkdirs();
        }
        // 开始写
        //BufferedOutputStream out = new BufferedOutputStream(new FileOutputStream(file.getAbsolutePath() + "/" +
originalFilename));
        // 可以采用UUID来生成文件名，防止服务器上传文件时产生覆盖
        BufferedOutputStream out = new BufferedOutputStream(new FileOutputStream(file.getAbsolutePath() + "/" +
UUID.randomUUID().toString() + originalFilename.substring(originalFilename.lastIndexOf("."))));
        byte[] bytes = new byte[1024 * 100];
        int readCount = 0;
        while((readCount = in.read(bytes)) != -1){
            out.write(bytes,0,readCount);
        }
        // 刷新缓冲流
        out.flush();
        // 关闭流
        in.close();
        out.close();

        return "ok";
    }
}

```

最终测试结果：

←
→
↻
localhost:8080/springmvc/

文件：

选择文件

1.jpeg

上传

上传成功



建议：上传文件时，文件起名采用UUID。以防文件覆盖。

文件下载

- index.html

```
<!-- 文件下载-->
<a th:href="@{/download}">文件下载</a>
```

文件下载核心程序，使用ResponseEntity：

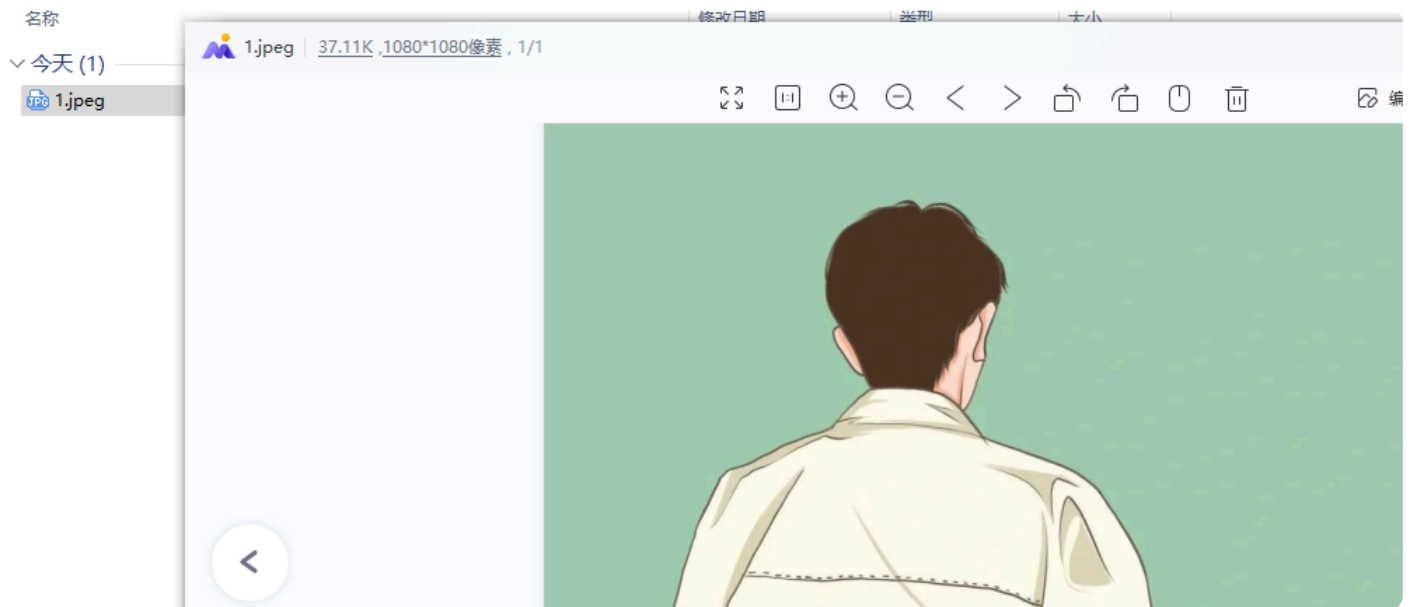
- FileController

```
@GetMapping("/download")
public ResponseEntity<byte[]> downloadFile(HttpServletResponse response, HttpServletRequest request) throws
IOException {
    File file = new File(request.getServletContext().getRealPath("/upload") + "/1.jpeg");
    // 创建响应头对象
    HttpHeaders headers = new HttpHeaders();
    // 设置响应内容类型
    headers.setContentType(MediaType.APPLICATION_OCTET_STREAM);
    // 设置下载文件的名称
    headers.setContentDispositionFormData("attachment", file.getName());

    // 下载文件
    ResponseEntity<byte[]> entity = new ResponseEntity<byte[]>(Files.readAllBytes(file.toPath()), headers,
    HttpStatus.OK);
    return entity;
}
```

效果：





异常处理器

什么是异常处理器

Spring MVC在处理器方法执行过程中出现了异常，可以采用异常处理器进行应对。

一句话概括异常处理器作用：处理器方法（Controller）执行过程中出现了异常，跳转到对应的视图，在视图上展示友好信息。

SpringMVC为异常处理提供了一个接口：HandlerExceptionResolver

```
public interface HandlerExceptionResolver {  
  
    /**  
     * Try to resolve the given exception that got thrown during handler execution, returning a  
     * ModelAndView that represents a specific error page if appropriate.  
     *  
     * The returned ModelAndView may be empty to indicate that the exception has been resolved  
     * successfully but that no view should be rendered, for instance by setting a status code.  
     *  
     * Params: request – current HTTP request  
     *         response – current HTTP response  
     *         handler – the executed handler, or null if none chosen at the time of the exception (for  
     *         example, if multipart resolution failed)  
     *         ex – the exception that got thrown during handler execution  
     *  
     * Returns: a corresponding ModelAndView to forward to, or null for default processing in the  
     *         resolution chain  
     */  
    @Nullable  
    ModelAndView resolveException(  
        HttpServletRequest request, HttpServletResponse response, @Nullable Object handler, Exception ex);  
}
```

核心方法是：resolveException。

该方法用来编写具体的异常处理方案。返回值ModelAndView，表示异常处理完之后跳转到哪个视图。

HandlerExceptionResolver 接口有两个常用的默认实现：

- DefaultHandlerExceptionResolver
- SimpleMappingExceptionResolver

默认异常处理器

DefaultHandlerExceptionResolver 是默认异常处理器。

核心方法：

```
@Override
@Nullable
protected ModelAndView doResolveException(
    HttpServletRequest request, HttpServletResponse response, @Nullable Object handler, Exception ex) {

    try {
        // ErrorResponse exceptions that expose HTTP response details
        if (ex instanceof ErrorResponse errorResponse) {
            ModelAndView mav = null;
            if (ex instanceof HttpRequestMethodNotSupportedException theEx) {
                mav = handleHttpRequestMethodNotSupported(theEx, request, response, handler);
            }
            else if (ex instanceof HttpMediaTypeNotSupportedException theEx) {
                mav = handleHttpMediaTypeNotSupported(theEx, request, response, handler);
            }
        }
    }
}
```

当请求方式和处理方式不同时，DefaultHandlerExceptionResolver的默认处理态度是：



自定义的异常处理器

自定义异常处理器需要使用：SimpleMappingExceptionResolver

自定义异常处理机制有两种语法：

- 通过XML配置文件
- 通过注解

配置文件方式

- springmvc.xml

```
<bean class="org.springframework.web.servlet.handler.SimpleMappingExceptionResolver">
    <property name="exceptionMappings">
        <props>
            <!-- 用来指定出现异常后，跳转的视图-->
            <prop key="java.lang.Exception">tip</prop>
        </props>
    </property>
    <!-- 将异常信息存储到request域，value属性用来指定存储时的key。-->
    <property name="exceptionAttribute" value="e"/>
</bean>
```

在视图页面上展示异常信息：

- tip.html



```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>出错了</title>
</head>
<body>
<h1>出错了，请联系管理员！</h1>
<div th:text="${e}"></div>
</body>
</html>
```

← → ↺ 🏠 ⓘ localhost:8080/springmvc/test

出错了，请联系管理员！

org.springframework.web.HttpRequestMethodNotSupportedException: Request method 'GET' is not supported

注解方式



```
@ControllerAdvice
public class ExceptionController {

    @ExceptionHandler
    public String tip(Exception e, Model model){
        model.addAttribute("e", e);
        return "tip";
    }

}
```

实际开发中的全局异常处理器

全局异常处理器的作用：统一处理应用中的异常，将异常信息转换为用户友好的响应，避免异常直接暴露给用户。

全局体现在：它能处理整个Spring MVC应用中所有控制器（Controller）抛出的异常，无需在每个控制器中单独编写异常处理代码。

第一步：自定义业务异常：



```
/**
 * 自定义业务异常类
 */
public class BusinessException extends RuntimeException {

    private String code;    // 错误码
    private String message; // 错误信息

    // 构造方法
    public BusinessException(String code, String message) {
        super(message);
        this.code = code;
        this.message = message;
    }

    public BusinessException(String code, String message, Throwable cause) {
        super(message, cause);
    }
}
```

```

        this.code = code;
        this.message = message;
    }

    // Getter 方法
    public String getCode() {
        return code;
    }

    public String getMessage() {
        return message;
    }
}

```

第二步：编写全局异常处理器

```

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.servlet.NoHandlerFoundException;

@ControllerAdvice
public class GlobalExceptionHandler{

    // 处理业务异常 - 页面跳转
    @ExceptionHandler(BusinessException.class)
    public String handleBusinessException(BusinessException e, Model model) {
        // 使用标准输出记录日志（生产环境建议使用日志框架）
        System.err.println("业务异常: code=" + e.getCode() + ", message=" + e.getMessage());
        e.printStackTrace();

        model.addAttribute("errorCode", e.getCode());
        model.addAttribute("errorMsg", e.getMessage());
        model.addAttribute("timestamp", java.time.LocalDateTime.now());

        return "error/business";
    }

    // 处理空指针异常
    @ExceptionHandler(NullPointerException.class)
    public String handleNullPointer(NullPointerException e, Model model) {
        System.err.println("空指针异常:");
        e.printStackTrace();

        model.addAttribute("error", "系统内部错误，请联系管理员");
        return "error/500";
    }

    // 处理所有其他异常（兜底）
    @ExceptionHandler(Exception.class)
    public String handleAllExceptions(Exception e, Model model, HttpServletRequest request) {
        System.err.println("未捕获异常: URI=" + request.getRequestURI());
        e.printStackTrace();

        model.addAttribute("error", "系统繁忙，请稍后重试");
        model.addAttribute("timestamp", java.time.LocalDateTime.now());
        model.addAttribute("path", request.getRequestURI());

        return "error/500";
    }

    // 处理404异常（需要配置）

```

```

@ExceptionHandler(NoHandlerFoundException.class)
public String handleNotFound(NoHandlerFoundException e, Model model) {
    System.err.println("404 - 页面未找到: " + e.getRequestURL());

    model.addAttribute("error", "请求的页面不存在");
    model.addAttribute("path", e.getRequestURL());
    return "error/404";
}
}

```

拦截器 Interceptor

拦截器概述

拦截器（Interceptor）类似于过滤器（Filter）

Spring MVC的拦截器作用是在请求到达控制器之前或之后进行拦截，可以对请求和响应进行一些特定的处理。

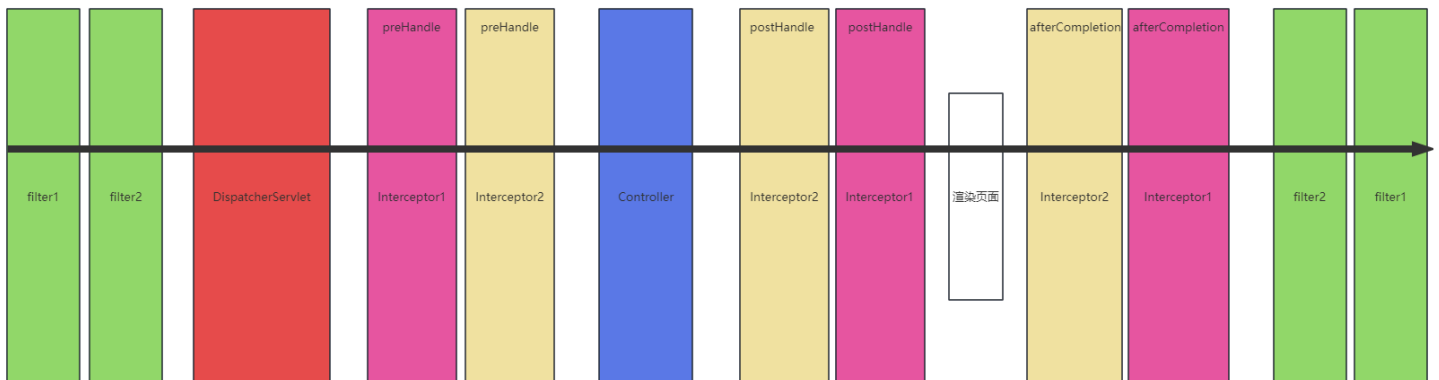
拦截器可以用于很多场景下：

1. 登录验证：对于需要登录才能访问的网址，使用拦截器可以判断用户是否已登录，如果未登录则跳转到登录页面。
2. 权限校验：根据用户权限对部分网址进行访问控制，拒绝未经授权的用户访问。
3. 请求日志：记录请求信息，例如请求地址、请求参数、请求时间等，用于排查问题和性能优化。
4. 更改响应：可以对响应的内容进行修改，例如添加头信息、调整响应内容格式等。

拦截器和过滤器的区别在于它们的作用层面不同。

- 过滤器更注重在请求和响应的流程中进行处理，可以修改请求和响应的内容，例如设置编码和字符集、请求头、状态码等。
- 拦截器则更加侧重于对控制器进行前置或后置处理，在请求到达控制器之前或之后进行特定的操作，例如打印日志、权限验证等。

Filter、Servlet、Interceptor、Controller的执行顺序：



面试题：过滤器和拦截器有什么区别？

过滤器是Servlet规范里的，工作在Tomcat这一层，所有请求不管是静态资源还是动态接口都得先过它这关，像是整个小区的门卫。拦截器是Spring MVC框架里的，只针对Controller层起作用，像是具体楼栋的保安。过滤器先执行，可以做全局的编码处理、安全过滤；拦截器后执行，能获取Spring容器中的Bean，更适合做权限校验、日志记录这类业务逻辑。

拦截器的创建与基本配置

定义拦截器

实现 `org.springframework.web.servlet.HandlerInterceptor` 接口，共有三个方法可以选择性的实现：

- `preHandle`：处理器方法调用之前执行
- 只有该方法有返回值，返回值是布尔类型，`true`放行，`false`拦截。
- `postHandle`：处理器方法调用之后执行
- `afterCompletion`：渲染完成后执行

拦截器基本配置

在springmvc.xml文件中进行如下配置：

第一种方式：

```
<mvc:interceptors>
  <bean class="com.jkweilai.springmvc.interceptors.Interceptor1"/>
</mvc:interceptors>
```

第二种方式：

```
<mvc:interceptors>
  <ref bean="interceptor1"/>
</mvc:interceptors>
```

第二种方式的前提：

- 前提1：包扫描，要保证能扫描到 `Interceptor1`
- 前提2：使用 `@Component` 注解进行标注

```
@Component
public class Interceptor1 implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest request, Http
```

注意：对于这种基本配置来说，拦截器是拦截所有请求的。

拦截器部分源码分析

方法执行顺序的源码分析

- DispatcherServlet部分源码

```
public class DispatcherServlet extends FrameworkServlet {
    protected void doDispatch(HttpServletRequest request, HttpServletResponse response) throws Exception {
        // 调用所有拦截器的 preHandle 方法
        if (!mappedHandler.applyPreHandle(processedRequest, response)) {
            return;
        }
        // 调用处理器方法
        mv = ha.handle(processedRequest, response, mappedHandler.getHandler());
        // 调用所有拦截器的 postHandle 方法
        mappedHandler.applyPostHandle(processedRequest, response, mv);
    }
}
```

```

// 处理视图
processDispatchResult(processedRequest, response, mappedHandler, mv, dispatchException);
}

private void processDispatchResult(HttpServletRequest request, HttpServletResponse response,
    @Nullable HandlerExecutionChain mappedHandler, @Nullable ModelAndView mv,
    @Nullable Exception exception) throws Exception {
    // 渲染页面
    render(mv, request, response);
    // 调用所有拦截器的 afterCompletion 方法
    mappedHandler.triggerAfterCompletion(request, response, null);
}
}

```

拦截与放行的源码分析

- DispatcherServlet部分源码

```

public class DispatcherServlet extends FrameworkServlet {
    protected void doDispatch(HttpServletRequest request, HttpServletResponse response) throws Exception {
        // 调用所有拦截器的 preHandle 方法
        if (!mappedHandler.applyPreHandle(processedRequest, response)) {
            // 如果 mappedHandler.applyPreHandle(processedRequest, response) 返回false, 以下的return语句就会执行
            return;
        }
    }
}

```

- HandlerExecutionChain部分源码

```

public class HandlerExecutionChain {
    boolean applyPreHandle(HttpServletRequest request, HttpServletResponse response) throws Exception {
        for (int i = 0; i < this.interceptorList.size(); i++) {
            HandlerInterceptor interceptor = this.interceptorList.get(i);
            if (!interceptor.preHandle(request, response, this.handler)) {
                triggerAfterCompletion(request, response, null);
                // 如果 interceptor.preHandle(request, response, this.handler) 返回 false, 以下的 return false; 就会执
                return false;
            }
            this.interceptorIndex = i;
        }
        return true;
    }
}

```

拦截器的高级配置

采用以上基本配置方式，拦截器是拦截所有请求路径的。如果要针对某些路径进行拦截，某些路径不拦截，可以采用高级配置：

- spring.xml

```

<mvc:interceptors>
  <mvc:interceptor>
    <!-- 拦截所有路径-->
    <mvc:mapping path="/**"/>
    <!-- 除 /test 路径之外-->
    <mvc:exclude-mapping path="/test"/>
    <!-- 拦截器-->
    <ref bean="interceptor1"/>
  </mvc:interceptor>
</mvc:interceptors>

```

以上的配置表示，除 /test 请求路径之外，剩下的路径全部拦截。

拦截器的执行顺序

执行顺序

如果所有拦截器preHandle都返回true

按照springmvc.xml文件中配置的顺序，自上而下调用 preHandle：

```

<mvc:interceptors>
  <ref bean="interceptor1"/>
  <ref bean="interceptor2"/>
</mvc:interceptors>

```

执行顺序：

```

Interceptor1's preHandle
Interceptor2's preHandle
Interceptor2's postHandle
Interceptor1's postHandle
Interceptor2's afterCompletion
Interceptor1's afterCompletion

```

如果其中一个拦截器preHandle返回false

```

<mvc:interceptors>
  <ref bean="interceptor1"/>
  <ref bean="interceptor2"/>
</mvc:interceptors>

```

如果 **interceptor2** 的preHandle返回false，执行顺序：

```

Interceptor1's preHandle
Interceptor2's preHandle
Interceptor1's afterCompletion

```

规则：只要有一个拦截器 **preHandle** 返回false，任何 **postHandle** 都不执行。但返回false的拦截器的前面的拦截器按照逆序执行 **afterCompletion**。

源码分析

DispatcherServlet和 HandlerExecutionChain的部分源码：

- DispatcherServlet部分源码

```
public class DispatcherServlet extends FrameworkServlet {
    protected void doDispatch(HttpServletRequest request, HttpServletResponse response) throws Exception {
        // 按照顺序执行所有拦截器的preHandle方法
        if (!mappedHandler.applyPreHandle(processedRequest, response)) {
            return;
        }
        // 执行处理器方法
        mv = ha.handle(processedRequest, response, mappedHandler.getHandler());
        // 按照逆序执行所有拦截器的 postHandle 方法
        mappedHandler.applyPostHandle(processedRequest, response, mv);
        // 处理视图
        processDispatchResult(processedRequest, response, mappedHandler, mv, dispatchException);
    }

    private void processDispatchResult(HttpServletRequest request, HttpServletResponse response,
        @Nullable HandlerExecutionChain mappedHandler, @Nullable ModelAndView mv,
        @Nullable Exception exception) throws Exception {
        // 渲染视图
        render(mv, request, response);
        // 按照逆序执行所有拦截器的 afterCompletion 方法
        mappedHandler.triggerAfterCompletion(request, response, null);
    }
}
```

- HandlerExecutionChain部分源码

```
public class HandlerExecutionChain {
    // 顺序执行 preHandle
    boolean applyPreHandle(HttpServletRequest request, HttpServletResponse response) throws Exception {
        for (int i = 0; i < this.interceptorList.size(); i++) {
            HandlerInterceptor interceptor = this.interceptorList.get(i);
            if (!interceptor.preHandle(request, response, this.handler)) {
                // 如果其中一个拦截器preHandle返回false
                // 将该拦截器前面的拦截器按照逆序执行所有的afterCompletion
                triggerAfterCompletion(request, response, null);
                return false;
            }
            this.interceptorIndex = i;
        }
        return true;
    }
    // 逆序执行 postHandle
    void applyPostHandle(HttpServletRequest request, HttpServletResponse response, @Nullable ModelAndView mv) throws Exception {
        for (int i = this.interceptorList.size() - 1; i >= 0; i--) {
            HandlerInterceptor interceptor = this.interceptorList.get(i);
            interceptor.postHandle(request, response, this.handler, mv);
        }
    }
    // 逆序执行 afterCompletion
    void triggerAfterCompletion(HttpServletRequest request, HttpServletResponse response, @Nullable Exception ex) {
        for (int i = this.interceptorIndex; i >= 0; i--) {
            HandlerInterceptor interceptor = this.interceptorList.get(i);
            try {
                interceptor.afterCompletion(request, response, this.handler, ex);
            }
        }
    }
}
```

```
        catch (Throwable ex2) {  
            logger.error("HandlerInterceptor.afterCompletion threw exception", ex2);  
        }  
    }  
}
```

SpringMVC 执行流程

SpringMVC 九大核心角色

前端控制器 (DispatcherServlet)

- 角色：中央调度器、总指挥
- 职责：接收所有请求，统一分发和协调

处理器映射器 (HandlerMapping)

- 角色：路由导航员
- 职责：根据请求URL找到对应的处理器（建立URL→处理器的映射）

处理器执行链 (HandlerExecutionChain)

- 角色：执行任务包
- 职责：封装处理器方法 + 拦截器集合，形成完整的执行单元

处理器 (Handler)

- 角色：业务执行者（通常是Controller）
- 职责：执行业务逻辑，处理具体请求

处理器适配器 (HandlerAdapter)

- 角色：万能转换器
- 职责：适配不同处理器类型，让DispatcherServlet能用统一方式调用它们

拦截器 (Interceptor)

- 角色：AOP切面卫士
- 职责：在处理器前后插入通用逻辑（权限、日志等）

ModelAndView

- 角色：数据视图封装器
- 职责：携带业务数据 (Model) + 视图信息 (View)

视图解析器 (ViewResolver)

- 角色：视图定位器
- 职责：将逻辑视图名（如"home"）解析为具体视图对象（如home.jsp）

视图 (View)

- 角色：页面渲染器
- 职责：最终渲染HTML/JSON等响应内容

从源码角度看执行流程

以下是核心代码：

- DispatcherServlet

```
public class DispatcherServlet extends FrameworkServlet {
    protected void doDispatch(HttpServletRequest request, HttpServletResponse response) throws Exception {
        // 根据请求对象获取处理器执行链
        // 处理器执行链中封装了：处理器方法 和 拦截器集合
        HandlerExecutionChain mappedHandler = getHandler(processedRequest); // 该方法底层调用了处理器映射器（根据请求路径
        找到对应处理器方法）

        // 通过处理器执行链获取处理器
        // 根据处理器获取适合的处理器适配器
        // 为什么需要一个处理器适配器？
        // 因为处理器类型多样（Controller、HttpRequestHandler等），适配器模式让DispatcherServlet能用统一方式调用它们。
        HandlerAdapter ha = getHandlerAdapter(mappedHandler.getHandler());

        // 顺序执行所有拦截器中的 preHandle 方法
        if (!mappedHandler.applyPreHandle(processedRequest, response)) {
            return;
        }

        // 通过处理器适配器调用处理器方法
        // 在调用处理器方法之前会进行数据绑定，将表单提交的数据绑定到处理器方法上。（底层是通过WebDataBinder完成的）
        // 如果有@RequestBody注解，则数据绑定的过程中会使用到消息转换器：HttpMessageConverter
        // 结束后返回ModelAndView对象
        mv = ha.handle(processedRequest, response, mappedHandler.getHandler());

        // 逆序执行所有拦截器中的 postHandle 方法
        mappedHandler.applyPostHandle(processedRequest, response, mv);

        // 处理分发结果（在这个方法中完成了响应）
        processDispatchResult(processedRequest, response, mappedHandler, mv, dispatchException);
    }

    // 根据每一次的请求对象来获取处理器执行链对象
    // 底层就是通过处理器映射器找到处理器方法，并将处理器方法（HandlerMethod）和拦截器集合封装为处理器执行链对象。
    protected HandlerExecutionChain getHandler(HttpServletRequest request) throws Exception {
        if (this.handlerMappings != null) {
            for (HandlerMapping mapping : this.handlerMappings) {
                HandlerExecutionChain handler = mapping.getHandler(request);
                if (handler != null) {
                    return handler;
                }
            }
        }
        return null;
    }

    private void processDispatchResult(HttpServletRequest request, HttpServletResponse response,
        @Nullable HandlerExecutionChain mappedHandler, @Nullable ModelAndView mv,
        @Nullable Exception exception) throws Exception {
        // 渲染
        render(mv, request, response);
        // 渲染完毕后，调用该请求对应的所有拦截器的 afterCompletion方法。
        mappedHandler.triggerAfterCompletion(request, response, null);
    }

    protected void render(ModelAndView mv, HttpServletRequest request, HttpServletResponse response) throws Exception
    {
        // 通过视图解析器返回视图对象
    }
```

```

        view = resolveViewName(viewName, mv.getModelInternal(), locale, request);
        // 真正的渲染视图
        view.render(mv.getModelInternal(), request, response);
    }

    protected View resolveViewName(String viewName, @Nullable Map<String, Object> model,
        Locale locale, HttpServletRequest request) throws Exception {
        // 通过视图解析器返回视图对象
        View view = viewResolver.resolveViewName(viewName, locale);
    }
}

```

- ViewResolver

```

public interface ViewResolver {
    View resolveViewName(String viewName, Locale locale) throws Exception;
}

```

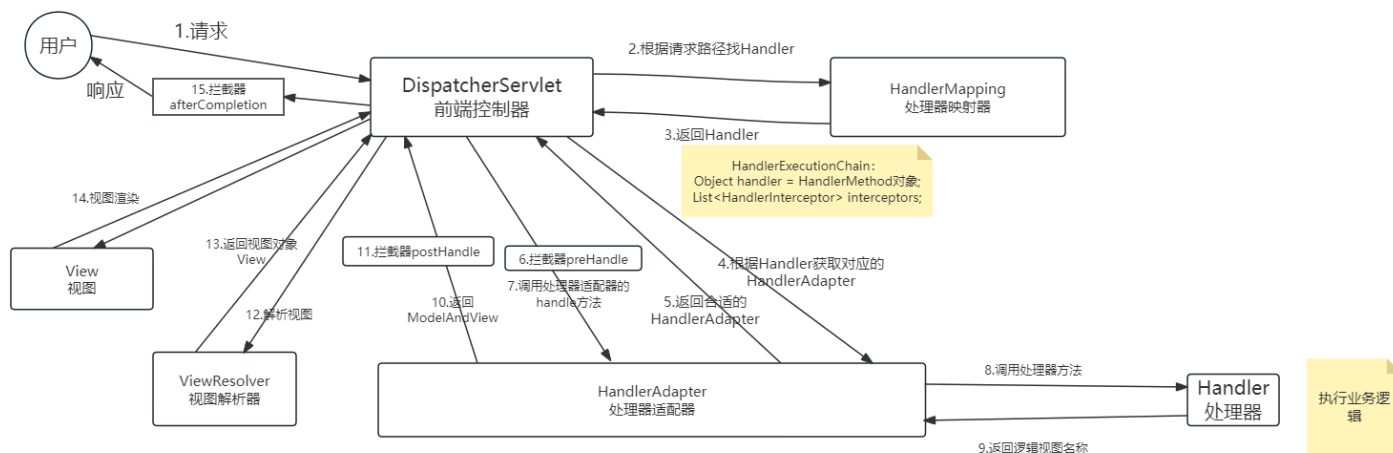
- View

```

public interface View {
    void render(@Nullable Map<String, ?> model, HttpServletRequest request, HttpServletResponse response)
        throws Exception;
}

```

从画图角度看执行流程



SpringMVC 执行流程文字性的描述:

1. 用户发送请求到 **前端控制器 (DispatcherServlet)**
2. 控制器调用 **doDispatch()** 方法进行统一分发
3. 通过 **处理器映射器 (HandlerMapping)** 查找对应的 **处理器执行链 (HandlerExecutionChain)**
4. 执行链包含: **处理器方法 (HandlerMethod)** + **拦截器列表 (Interceptors)**
5. 根据 **处理器类型** 获取对应的 **处理器适配器 (HandlerAdapter)**
6. **顺序执行** 所有拦截器的 **preHandle()** 方法
7. 适配器调用处理器的目标方法执行业务逻辑
8. 处理器返回 **逻辑视图名** 给适配器
9. 适配器封装成 **ModelAndView** 返回给前端控制器

10. 逆序执行拦截器的 `postHandle()` 方法
11. 前端控制器调用 视图解析器 (`ViewResolver`) 解析视图 (View)
12. 视图(View)进行渲染并返回响应
13. 逆序执行拦截器的 `afterCompletion()` 方法 (无论成功失败都会执行)

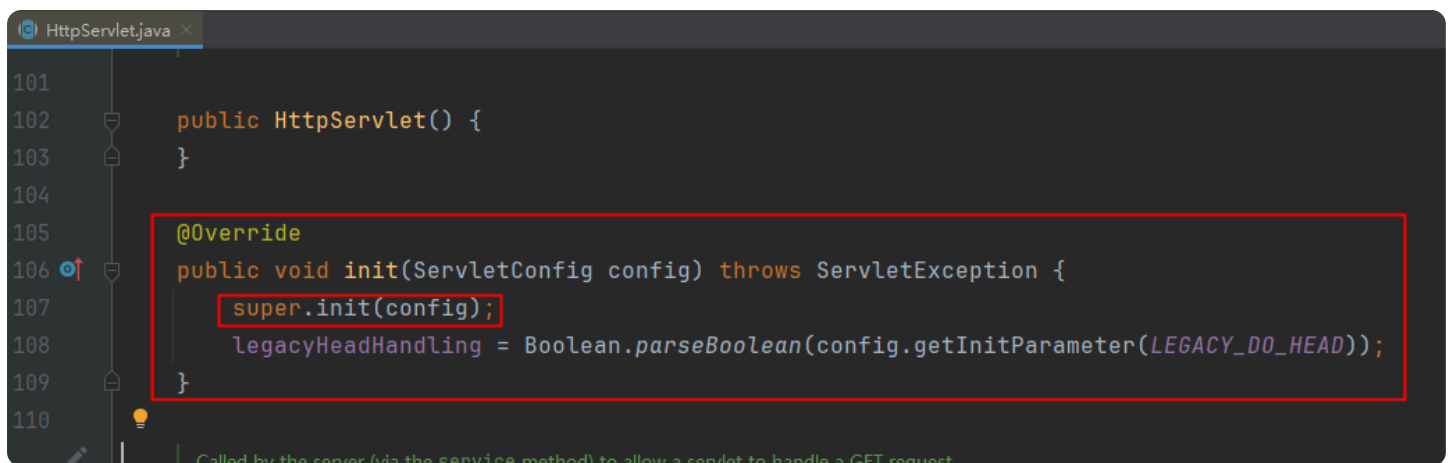
处理器映射器和处理器适配器的创建时机

先搞明白核心类的继承关系：

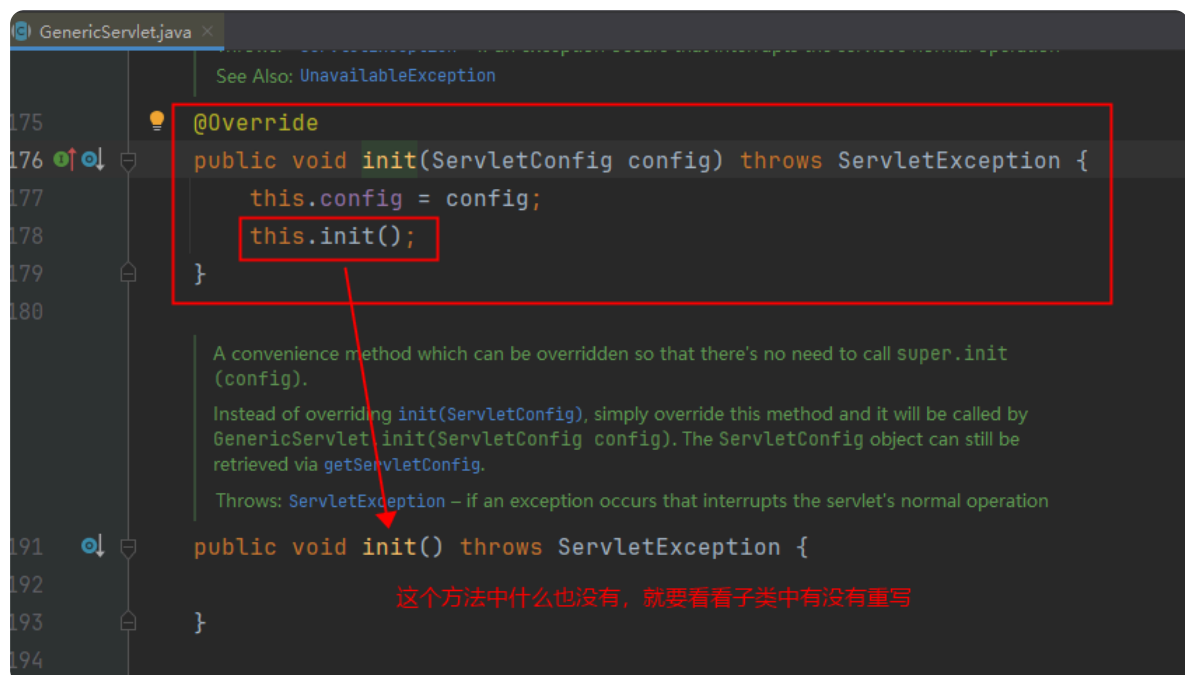
DispatcherServlet extends FrameworkServlet extends HttpServletBean extends HttpServlet extends GenericServlet implements Servlet

服务器启动阶段完成了：

1. 初始化Spring上下文，也就是创建所有的bean，让IoC容器将其管理起来。
2. 初始化SpringMVC相关的对象：处理器映射器，处理器适配器等。。。



```
101
102 public HttpServlet() {
103 }
104
105 @Override
106 public void init(ServletConfig config) throws ServletException {
107     super.init(config);
108     legacyHeadHandling = Boolean.parseBoolean(config.getInitParameter(LEGACY_DO_HEAD));
109 }
110
```



```
175
176 @Override
177 public void init(ServletConfig config) throws ServletException {
178     this.config = config;
179     this.init();
180 }
181
182 A convenience method which can be overridden so that there's no need to call super.init
183 (config).
184 Instead of overriding init(ServletConfig), simply override this method and it will be called by
185 GenericServlet.init(ServletConfig config). The ServletConfig object can still be
186 retrieved via getServletConfig.
187 Throws: ServletException – if an exception occurs that interrupts the servlet's normal operation
188
189 public void init() throws ServletException {
190 }
191
192
193
194
```

这个方法中什么也没有，就要看看子类中有没有重写

HttpServletBean.java

throws ServletException – if bean properties are invalid (or required properties are missing), or if subclass initialization fails.

```
147  @Override
148  public final void init() throws ServletException {
149
150      // Set bean properties from init parameters.
151      PropertyValues pvs = new ServletConfigPropertyValues(getServletConfig(), this.requiredProperties);
152      if (!pvs.isEmpty()) {
153          try {
154              BeanWrapper bw = PropertyAccessorFactory.forBeanPropertyAccess(target: this);
155              ResourceLoader resourceLoader = new ServletContextResourceLoader(getServletContext());
156              bw.registerCustomEditor(Resource.class, new ResourceEditor(resourceLoader, getEnvironment()));
157              initBeanWrapper(bw);
158              bw.setPropertyValues(pvs, ignoreUnknown: true);
159          }
160          catch (BeansException ex) {
161              if (logger.isDebugEnabled()) {
162                  logger.error("message: \"Failed to set bean properties on servlet '\" + getServletName() + '\"\", ex);
163              }
164              throw ex;
165          }
166      }
167
168      // Let subclasses do whatever initialization they like.
169      initServletBean();
170  }
```

FrameworkServlet.java

Override method of HttpServletBean, invoked after any bean properties have been set. Creates this servlet's WebApplicationContext.

```
528  @Override
529  protected final void initServletBean() throws ServletException {
530      getServletContext().log(msg: "Initializing Spring " + getClass().getSimpleName() + " '" + getServletName() + '"');
531      if (logger.isDebugEnabled()) {
532          logger.info("Initializing Servlet '" + getServletName() + '"');
533      }
534      long startTime = System.currentTimeMillis();
535
536      try {
537          this.webApplicationContext = initWebApplicationContext();
538          initFrameworkServlet();
539      }
540      catch (ServletException | RuntimeException ex) {
541          logger.error("message: \"Context initialization failed\", ex);
542          throw ex;
543      }
544
545      if (logger.isDebugEnabled()) {
546          String value = this.enableLoggingRequestDetails ?
547              "shown which may lead to unsafe logging of potentially sensitive data" :
548              "masked to prevent unsafe logging of potentially sensitive data";
549          logger.debug("enableLoggingRequestDetails='" + this.enableLoggingRequestDetails +
550              "': request parameters and headers will be " + value);
551      }
552
553      if (logger.isDebugEnabled()) {
554          logger.info("Completed initialization in " + (System.currentTimeMillis() - startTime) + " ms");
555      }
556  }
```

```

FrameworkServlet.java x
See Also: FrameworkServlet(WebApplicationContext),
              setContextClass,
              setContextConfigLocation
567     protected WebApplicationContext initWebApplicationContext() {
568         WebApplicationContext rootContext =
569             WebApplicationContextUtils.getWebApplicationContext(g
570         WebApplicationContext wac = null;
571
572         if (this.webApplicationContext != null) {

```

```

FrameworkServlet.java x
598         if (!this.refreshEventReceived) {
599             // Either the context is not a ConfigurableApplicationContext
600             // support or the context injected at construction time had
601             // refreshed -> trigger initial onRefresh manually here.
602             synchronized (this.onRefreshMonitor) {
603                 onRefresh(wac);
604             }
605         }

```

```

FrameworkServlet.java x
This implementation is empty.
Params: context – the current WebApplicationContext
See Also: refresh()
857     protected void onRefresh(ApplicationContext context) {
858         // For subclasses: do nothing by default.
859     }

```

```

DispatcherServlet.java x
This implementation calls initStrategies.
500     @Override
501     protected void onRefresh(ApplicationContext context) {
502         initStrategies(context);
503     }
504
505     Initialize the strategy objects that this servlet uses.
506     May be overridden in subclasses in order to initialize further strategy objects.
509     protected void initStrategies(ApplicationContext context) {
510         initMultipartResolver(context);
511         initLocaleResolver(context);
512         initThemeResolver(context);
513         initHandlerMappings(context);
514         initHandlerAdapters(context);
515         initHandlerExceptionResolvers(context);
516         initRequestToViewNameTranslator(context);
517         initViewResolvers(context);
518         initFlashMapManager(context);
519     }

```

常见面试题

Q：处理器映射器和适配器什么时候创建？

在Spring容器启动时，DispatcherServlet初始化阶段创建的。具体是在 `initStrategies()` 方法中，通过 `initHandlerMappings()` 和 `initHandlerAdapters()` 方法从Spring容器中获取所有相关Bean，并按优先级排序后缓存在DispatcherServlet中。这样请求到来时就能直接使用，不需要每次请求都重新创建。

Q：请你描述一下 SpringMVC 的执行流程？

1. 用户发送请求到 **前端控制器（DispatcherServlet）**
2. 控制器调用 `doDispatch()` 方法进行统一分发
3. 通过 **处理器映射器（HandlerMapping）** 查找对应的 **处理器执行链（HandlerExecutionChain）**
4. 执行链包含：**处理器方法（HandlerMethod）** + **拦截器列表（Interceptors）**
5. 根据处理器类型获取对应的 **处理器适配器（HandlerAdapter）**
6. **顺序执行** 所有拦截器的 `preHandle()` 方法
7. 适配器调用处理器的目标方法执行业务逻辑
8. 处理器返回 **逻辑视图名** 给适配器
9. 适配器封装成 **ModelAndView** 返回给前端控制器
10. **逆序执行** 拦截器的 `postHandle()` 方法
11. 前端控制器调用 **视图解析器（ViewResolver）** 解析视图
12. 视图进行渲染并返回响应
13. **逆序执行** 拦截器的 `afterCompletion()` 方法（抛异常了也会执行）

Q：DispatcherServlet的作用？

它是Spring MVC的总指挥，负责请求分发、协调各组件工作。

Q：为什么需要HandlerAdapter？

因为处理器类型多样（Controller、HttpRequestHandler等），适配器模式让DispatcherServlet能用统一方式调用它们。

Q：拦截器三个方法的执行时机？

`preHandle`在处理器前，`postHandle`在处理器后但视图渲染前，`afterCompletion`在视图渲染后。

Q：过滤器和拦截器在流程中的位置？

过滤器在最外层（Tomcat层面），然后才进入Spring MVC的DispatcherServlet → 拦截器 → 处理器。

SpringMVC 全注解开发

web.xml文件的替代

Servlet3.0新特性

Servlet3.0新特性：web.xml文件可以不写了。
在Servlet3.0的时候，规范中提供了一个接口：

```
services are discovered must follow the application's classloading delegation model.
Since: Servlet 3.0
See Also: jakarta.servlet.annotation.HandlesTypes

public interface ServletContainerInitializer {

    Notifies this ServletContainerInitializer of the startup of the application represented by the
    given ServletContext.

    If this ServletContainerInitializer is bundled in a JAR file inside the WEB-INF/lib directory of an
    application, its onStartUp method will be invoked only once during the startup of the bundling
    application. If this ServletContainerInitializer is bundled inside a JAR file outside of any WEB-
    INF/lib directory, but still discoverable as described above, its onStartUp method will be invoked
    every time an application is started.

    Params: c - the Set of application classes that extend, implement, or have been annotated with
    the class types specified by the HandlesTypes annotation, or null if there are no matches,
    or this ServletContainerInitializer has not been annotated with HandlesTypes
    ctx - the ServletContext of the web application that is being started and in which the
    classes contained in c were found

    Throws: ServletException - if an error has occurred

    public void onStartUp(Set<Class?>> c, ServletContext ctx) throws ServletException;
}
```

服务器在启动的时候会自动从容器中找到 `ServletContainerInitializer` 接口的实现类，自动调用它的 `onStartup` 方法来完成Servlet上下文的初始化。

在Spring3.1版本的时候，提供了这样一个类，实现以上的接口：

```
Since: 3.1
See Also: onStartUp(Set, ServletContext),
WebApplicationInitializer
Author: Chris Beams, Juergen Hoeller, Rossen Stoyanchev

@HandlesTypes(WebApplicationInitializer.class)
public class SpringServletContainerInitializer implements ServletContainerInitializer {

    Delegate the ServletContext to any WebApplicationInitializer implementations present on the
    application classpath.
```



它的核心方法如下：

```
@Override
public void onStartUp(@Nullable Set<Class?>> webAppInitializerClasses, ServletContext servletContext)
    throws ServletException {

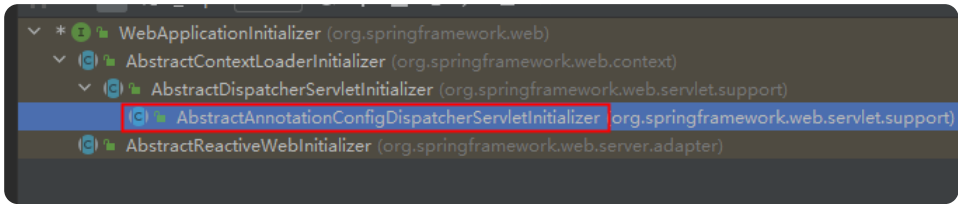
    List<WebApplicationInitializer> initializers = Collections.emptyList();

    if (webAppInitializerClasses != null) {
        initializers = new ArrayList<>(webAppInitializerClasses.size());
        for (Class?> waiClass : webAppInitializerClasses) {
            // Be defensive: Some servlet containers provide us with invalid classes,
            // no matter what @HandlesTypes says...
            if (!waiClass.isInterface() && !Modifier.isAbstract(waiClass.getModifiers()) &&
                WebApplicationInitializer.class.isAssignableFrom(waiClass)) {
                try {
                    initializers.add((WebApplicationInitializer)
                        ReflectionUtils.accessibleConstructor(waiClass).newInstance());
                }
                catch (Throwable ex) {
                    throw new ServletException("Failed to instantiate WebApplicationInitializer class", ex);
                }
            }
        }
    }
}
```

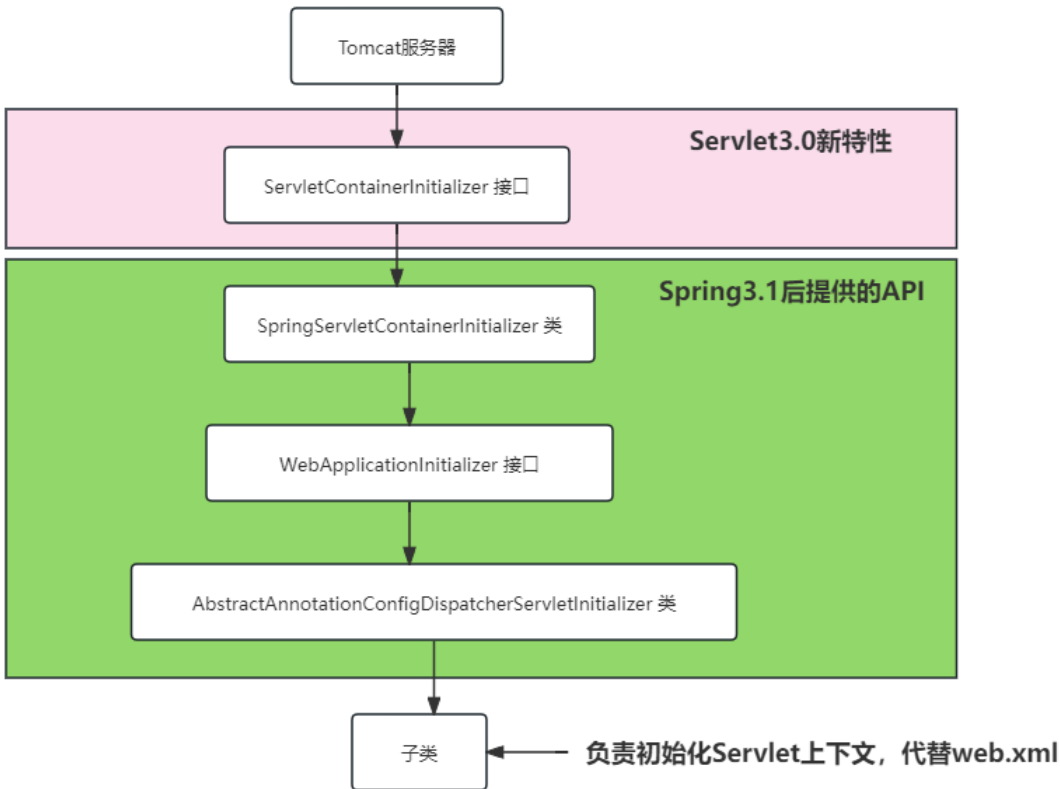
可以看到在服务器启动的时候，它会去加载所有实现 `WebApplicationInitializer` 接口的类：

```
on container initialization within a single WebApplicationInitializer.  
Since: 3.1  
See Also: SpringServletContainerInitializer,  
           org.springframework.web.context.AbstractContextLoaderInitializer,  
           org.springframework.web.servlet.support.AbstractDispatcherServletInitializer,  
           org.springframework.web.servlet.support.  
           AbstractAnnotationConfigDispatcherServletInitializer  
Author: Chris Beams  
  
public interface WebApplicationInitializer {  
  
    Configure the given ServletContext with any servlets, filters, listeners context-params and  
    attributes necessary for initializing this web application. See examples above.  
    Params: servletContext – the ServletContext to initialize  
    Throws: ServletException – if any call against the given ServletContext throws a  
           ServletException  
  
    void onStartUp(ServletContext servletContext) throws ServletException;  
  
}
```

这个接口下有一个子类是我们需要的： **AbstractAnnotationConfigDispatcherServletInitializer**



当我们编写类继承 **AbstractAnnotationConfigDispatcherServletInitializer** 之后，web服务器在启动的时候会根据它来初始化Servlet上下文。



编写WebAppInitializer

以下这个类就是用来代替web.xml文件的：

```
package com.jkweilai.springmvc.config;

import jakarta.servlet.Filter;
import org.springframework.web.filter.CharacterEncodingFilter;
import org.springframework.web.filter.HiddenHttpMethodFilter;
import org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializer;

// 以下配置是代替 web.xml 的。
// 等同于在web.xml文件中配置DispatcherServlet
// 1. 指定springmvc.xml文件的位置。
// 2. 指定在服务器启动时加载。（不需要指定，指定扫描包的时候，能扫描到即可）
// 3. 配置映射路径url-pattern
// 4. 字符编码过滤器
// 5. HiddenHttpMethodFilter

// 本身是一个 Servlet 容器初始化器，它的生命周期由 Servlet 容器（如Tomcat）管理，而不是由Spring容器管理。
// WebConfig 类的作用是：配置DispatcherServlet本身，它会在Spring容器启动之前就被Servlet容器调用。
// @Configuration // 不能添加 @Configuration 注解
public class WebConfig extends AbstractAnnotationConfigDispatcherServletInitializer {

    // 我们之前写的springmvc.xml文件中的配置，既有spring的配置，又有springmvc的配置
    // 中大型项目一般都将这两个配置分开，我们这里将springmvc.xml拆分为两个配置类。
    // 1. SpringConfig 类编写Spring配置。
    // 2. SpringMvcConfig 类编写SpringMVC配置。

    // 指定Spring的配置
    @Override
    protected Class<?>[] getRootConfigClasses() {
        return new Class[]{SpringConfig.class};
    }

    // 指定SpringMVC的配置
    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class[]{SpringMvcConfig.class};
    }

    // 配置DispatcherServlet的 url-pattern
    @Override
    protected String[] getServletMappings() {
        return new String[]{"/*"};
    }

    // 配置字符编码过滤器以及RESTful过滤器
    @Override
    protected Filter[] getServletFilters() {
        CharacterEncodingFilter characterEncodingFilter = new CharacterEncodingFilter();
        characterEncodingFilter.setEncoding("UTF-8");
        characterEncodingFilter.setForceRequestEncoding(true);
        characterEncodingFilter.setForceResponseEncoding(true);
        HiddenHttpMethodFilter hiddenHttpMethodFilter = new HiddenHttpMethodFilter();
        return new Filter[]{characterEncodingFilter, hiddenHttpMethodFilter};
    }
}
```

Spring配置如下：

```

package com.jkweilai.springmvc.config;

import org.springframework.context.annotation.Configuration;

@Configuration
public class SpringConfig {
}

```

SpringMVC配置如下：

```

package com.jkweilai.springmvc.config;

import org.springframework.context.annotation.Configuration;

@Configuration
public class SpringMvcConfig {
}

```

Spring 配置

组件扫描我们归纳到 Spring 的配置下：

```

package com.jkweilai.springmvc.config;

import org.springframework.context.annotation.Configuration;

// 以后这里配置Spring相关的配置，例如：数据源、事务管理等。
@Configuration
public class SpringConfig {
}

```

Spring MVC的配置

重点注意事项：SpringMVC 的配置类需要实现一个固定的接口 `WebMvcConfigurer`

开启注解驱动及扫描 controller

```

package com.jkweilai.springmvc.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
@EnableWebMvc // 代替: <mvc:annotation-driven/>
// 注意：所有controller的扫描需要放到 SpringMvcConfig 中，如果只放到 SpringConfig 中进行扫描是不行的。
@ComponentScan(basePackages = {"com.jkweilai.springmvc.controller"})
public class SpringMvcConfig implements WebMvcConfigurer {
}

```

使用了 `@EnableWebMvc`，你的类 **必须实现** `WebMvcConfigurer` 接口来配置静态资源处理，否则所有静态资源（CSS、JS、图片）都无法访问。

视图解析器

```
● ● ●
// 配置Thymeleaf视图解析器
@Bean
public ThymeleafViewResolver thymeleafViewResolver() {
    ThymeleafViewResolver viewResolver = new ThymeleafViewResolver();
    viewResolver.setTemplateEngine(templateEngine());
    viewResolver.setCharacterEncoding("UTF-8");
    viewResolver.setOrder(1);
    return viewResolver;
}

private SpringTemplateEngine templateEngine() {
    SpringTemplateEngine engine = new SpringTemplateEngine();
    engine.setTemplateResolver(templateResolver());
    return engine;
}

@Bean
public SpringResourceTemplateResolver templateResolver() {
    SpringResourceTemplateResolver resolver = new SpringResourceTemplateResolver();
    resolver.setPrefix("/WEB-INF/templates/");
    resolver.setSuffix(".html");
    resolver.setTemplateMode(TemplateMode.HTML);
    resolver.setCharacterEncoding("UTF-8");
    resolver.setCacheable(false);
    return resolver;
}
```

配置静态资源

```
● ● ●
// 静态资源配置
@Override
public void addResourceHandlers(ResourceHandlerRegistry registry) {
    registry.addResourceHandler("/static/**")
        .addResourceLocations("/static/")
        .setCachePeriod(3600);
}
```

配置 view-controller

```
● ● ●
// 配置视图控制器（对于没有业务的Controller可以直接配置，不需要写）
@Override
public void addViewControllers(ViewControllerRegistry registry) {
    registry.addViewController("/").setViewName("index");
}
```

配置拦截器

编写拦截器：

```
● ● ●
package com.jkweilai.springmvc.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.HandlerInterceptor;
```



```
public class SecurityInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) throws
Exception {
        System.out.println("拦截器执行，检查你有没有权限...");
        return true;
    }
}
```

```
package com.jkweilai.springmvc.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.HandlerInterceptor;

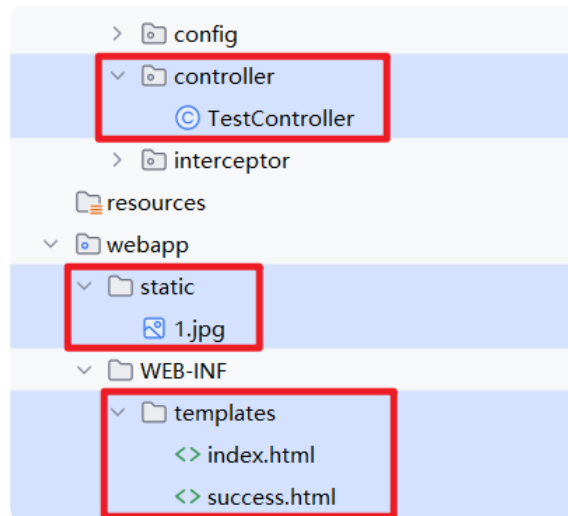
public class LogInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) throws
Exception {
        System.out.println("拦截器执行，我用来记录操作日志...");
        return true;
    }
}
```

配置拦截器：

```
// 配置拦截器
@Override
public void addInterceptors(InterceptorRegistry registry) {
    // 安全拦截器
    SecurityInterceptor securityInterceptor = new SecurityInterceptor();
    // 指定哪些路径拦截，哪些不拦截
    registry.addInterceptor(securityInterceptor).addPathPatterns("/**").excludePathPatterns("/test");
    // 日志拦截器
    LogInterceptor logInterceptor = new LogInterceptor();
    // 指定哪些路径拦截，哪些不拦截
    registry.addInterceptor(logInterceptor).addPathPatterns("/**").excludePathPatterns("/test");
}
```

编写测试程序进行测试

创建好对应的文件、目录及程序：



- index.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>index page</title>
</head>
<body>
<h1>测试SpringMVC全注解开发</h1>
<h3><a th:href="@{/test}"></a></h3>
</body>
</html>
```

- success.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>index page</title>
</head>
<body>
<h1>success</h1>
</body>
</html>
```

- TestController

```
package com.jkweilai.springmvc.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class TestController {

    @GetMapping("/test")
    public String test(){
        return "success";
    }
}
```

访问首页面: <http://localhost:8080/springmvc/>

← → ↻ ⓘ localhost:8080/springmvc/

测试SpringMVC全注解开发



拦截器执行, 检查你有没有权限...

拦截器执行, 我用来记录操作日志...

拦截器执行, 检查你有没有权限...

拦截器执行, 我用来记录操作日志...

再发送 /test请求: <http://localhost:8080/springmvc/test>

← → ↻ ⓘ localhost:8080/springmvc/test

success

SSM 整合-全注解式开发

在我们上面 SpringMVC 全注解开发的基础之上添加 Spring+MyBatis 的配置就行了。

我们在前面学习 Spring 的时候, 已经实现了 Spring+MyBatis 的全注解方式开发, 我们将当时的配置拿过来就行了。

第一步: 引入 SSM 依赖

```
<dependencies>
  <!--springmvc 依赖-->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>6.2.13</version>
  </dependency>
  <!--servlet 依赖-->
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.0.0</version>
    <scope>provided</scope>
  </dependency>
  <!--Lombok: 可选-->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.42</version>
```

```
        <scope>compile</scope>
    </dependency>
    <!-- thymeleaf 与 spring 整合依赖 -->
    <dependency>
        <groupId>org.thymeleaf</groupId>
        <artifactId>thymeleaf-spring6</artifactId>
        <version>3.1.3.RELEASE</version>
    </dependency>
    <!-- spring context -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>6.2.13</version>
    </dependency>
    <!-- AspectJ 依赖 -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-aspects</artifactId>
        <version>6.2.13</version>
    </dependency>
    <!-- spring jdbc -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-jdbc</artifactId>
        <version>6.2.13</version>
    </dependency>
    <!-- mysql 驱动 -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>8.4.0</version>
    </dependency>
    <!-- mybatis 依赖 -->
    <dependency>
        <groupId>org.mybatis</groupId>
        <artifactId>mybatis</artifactId>
        <version>3.5.16</version>
    </dependency>
    <!-- mybatis 和 spring 集成的依赖 -->
    <dependency>
        <groupId>org.mybatis</groupId>
        <artifactId>mybatis-spring</artifactId>
        <version>3.0.4</version>
    </dependency>
    <!-- HikariCP 连接池的依赖 -->
    <dependency>
        <groupId>com.zaxxer</groupId>
        <artifactId>HikariCP</artifactId>
        <version>7.0.2</version>
    </dependency>
    <!-- junit 的依赖 -->
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-api</artifactId>
        <version>5.11.0</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-test</artifactId>
        <version>6.2.13</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

第二步：编写 mybatis-config.xml 文件

在类的根路径下创建该配置文件：mybatis-config.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="LogImpl" value="STDOUT_LOGGING"/>
    </settings>
</configuration>
```

第三步：编写 application.properties 文件

在类的根路径下创建该属性配置文件：application.properties

```
spring.datasource.driver=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ssm
spring.datasource.user=root
spring.datasource.password=123456
mybatis.config.location=mybatis-config.xml
mybatis.typeAliases.package=com.jkweilai.ssm.entity
```

第四步：编写 Spring 和 MyBatis 的配置

我们之前编写 SpringMVC 全注解式开发的时候，写过一个 SpringConfig 配置类，将以下代码拷贝进去即可。

```
package com.jkweilai.ssm.config;

import com.zaxxer.hikari.HikariDataSource;
import org.mybatis.spring.SqlSessionFactoryBean;
import org.mybatis.spring.annotation.MapperScan;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;

@Configuration
@ComponentScan("com.jkweilai.ssm")
@PropertySource("classpath:application.properties")
@EnableTransactionManagement
@EnableAspectJAutoProxy
@MapperScan("com.jkweilai.ssm.mapper")
public class SpringConfig {

    @Bean
    public DataSource dataSource(
        @Value("${spring.datasource.driver}")
        String driver,
```

```

        @Value("${spring.datasource.url}")
        String url,
        @Value("${spring.datasource.user}")
        String user,
        @Value("${spring.datasource.password}")
        String password) {
    HikariDataSource dataSource = new HikariDataSource();
    dataSource.setDriverClassName(driver);
    dataSource.setJdbcUrl(url);
    dataSource.setUsername(user);
    dataSource.setPassword(password);
    return dataSource;
}

@Bean
public SqlSessionFactoryBean sqlSessionFactoryBean(
    DataSource dataSource,
    @Value("${mybatis.config.location}")
    String mybatisConfigLocation,
    @Value("${mybatis.typeAliases.package}")
    String typeAliasesPackage) {
    SqlSessionFactoryBean sqlSessionFactoryBean = new SqlSessionFactoryBean();
    Resource resource = new ClassPathResource(mybatisConfigLocation);
    sqlSessionFactoryBean.setConfigLocation(resource);
    sqlSessionFactoryBean.setDataSource(dataSource);
    sqlSessionFactoryBean.setTypeAliasesPackage(typeAliasesPackage);
    return sqlSessionFactoryBean;
}

@Bean
public DataSourceTransactionManager transactionManager(DataSource dataSource) {
    DataSourceTransactionManager dataSourceTransactionManager = new DataSourceTransactionManager();
    dataSourceTransactionManager.setDataSource(dataSource);
    return dataSourceTransactionManager;
}
}

```

第五步：编写测试程序

1. 准备数据库表

创建数据库 ssm，并且在该数据库中创建表 t_user

```

● ● ●
drop table if exists t_user;
create table t_user(
    id int primary key auto_increment,
    username varchar(255),
    password varchar(255)
);

```

2. 编写实体类

```

package com.jkweilai.ssm.entity;

import lombok.Data;

@Data
public class User {
    private Integer id;
    private String username;
    private String password;
}

```

3. 编写 SqlMapper.xml 文件

在 `resources` 目录下创建目录：`com/jkweilai/ssm/mapper`，并创建 `UserMapper.xml` 配置文件：

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.jkweilai.ssm.mapper.UserMapper">
    <insert id="insert">
        insert into t_user(username,password) values(#{username}, #{password})
    </insert>
</mapper>

```

4. 编写 Mapper 接口

```

package com.jkweilai.ssm.mapper;

import com.jkweilai.ssm.entity.User;

public interface UserMapper {
    int insert(User user);
}

```

5. 编写 service 接口和实现

```

package com.jkweilai.ssm.service;

import com.jkweilai.ssm.entity.User;

public interface UserService {
    int save(User user);
}

```

```

package com.jkweilai.ssm.service.impl;

import com.jkweilai.ssm.entity.User;
import com.jkweilai.ssm.mapper.UserMapper;
import com.jkweilai.ssm.service.UserService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor

```

```

public class UserServiceImpl implements UserService {

    private final UserMapper userMapper;

    @Override
    @Transactional // 添加事务控制
    public int save(User user) {
        return userMapper.insert(user);
    }
}

```

6. 编写 controller

```

package com.jkweilai.ssm.controller;

import com.jkweilai.ssm.entity.User;
import com.jkweilai.ssm.service.UserService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @PostMapping("/save")
    public String save(User user){
        userService.save(user);
        return "success";
    }
}

```

7. 编写页面

- index.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>index page</title>
</head>
<body>
<h1>SSM整合（全注解开发）</h1>
<form th:action="@{/save}" method="post">
    用户名: <input type="text" name="username">
    <br>
    密码: <input type="password" name="password">
    <br>
    <button>保存</button>
</form>
</body>
</html>

```

- success.html



```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>success</title>
</head>
<body>
<h1>保存成功</h1>
</body>
</html>
```

启动服务器，然后打开首页面，填写表单，保存数据，最终数据库中成功插入一条记录，则表示 SSM 整合成功。

到此，SpringMVC 的课程就结束了。